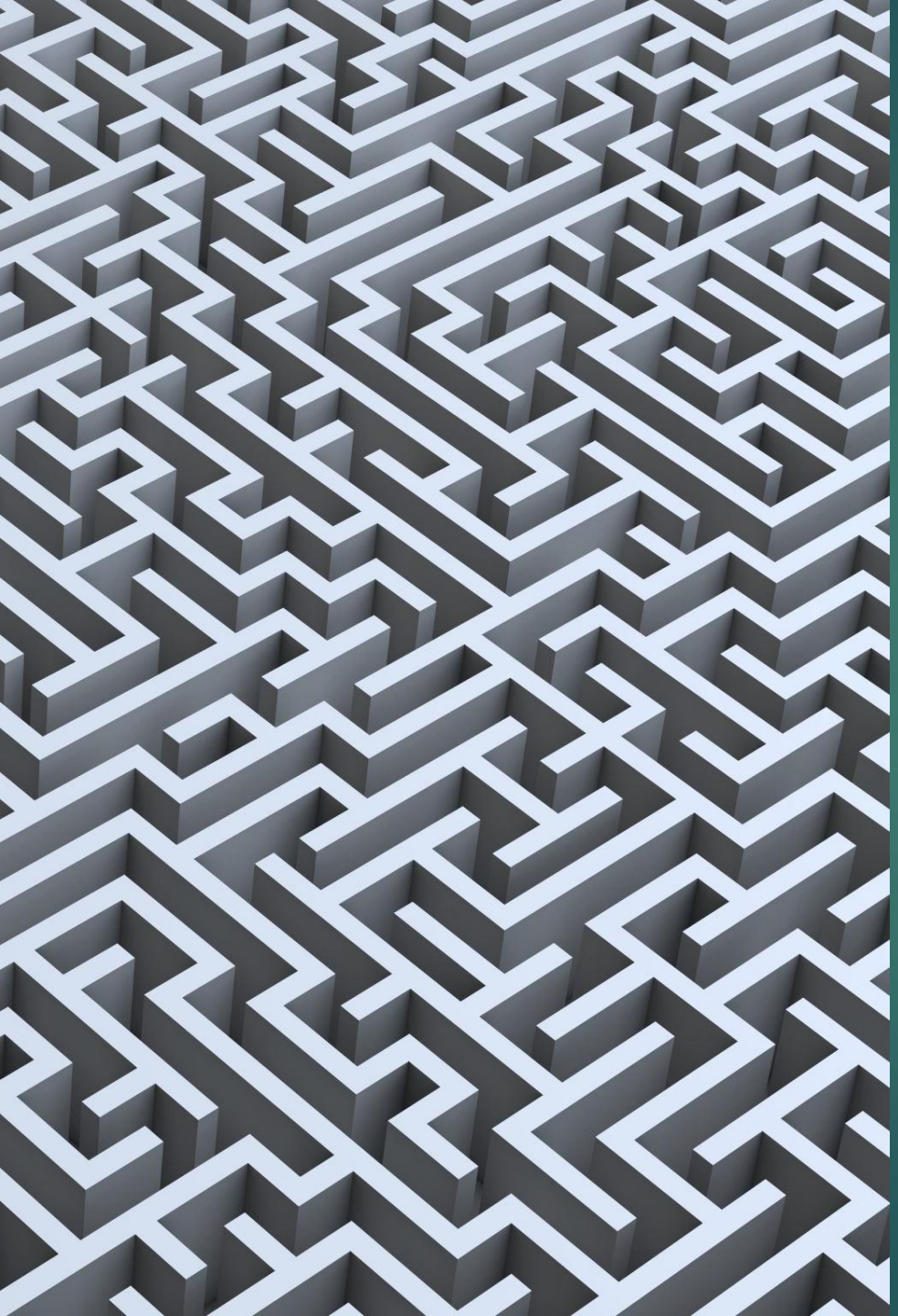
Projet maze : programmation impérative

MAZE RUNNER :



Projet Maze Runner :

Résolution de labyrinthes avec BFS et algorithme génétique

Réalisé par :

Younes Nassih (The Goat)

Prof encadrant : Bonnet Denis

ICE3 : 2025/2026

Dans ce projet, j’ai travaillé sur la génération et la résolution de labyrinthes en Python. J’ai commencé par créer des labyrinthes de manière aléatoire, puis j’ai développé une méthode fiable pour trouver automatiquement le chemin le plus court entre un point de départ et un point objectif. Cette méthode sert de référence pour évaluer les résultats. Ensuite, j’ai mis en place une approche différente basée sur un algorithme génétique, qui cherche progressivement de meilleurs chemins en testant et en améliorant des solutions au fil des générations. Le projet comprend également des visualisations et des tests permettant d’analyser le comportement des algorithmes et de comparer leurs performances.

SOUS_PROJET1 :

L’objectif de ce sous-projet est de générer des labyrinthes aléatoires de manière contrôlée et de les visualiser sous forme d’images afin de vérifier leur structure et de préparer les étapes de résolution.

Le labyrinthe est représenté par une matrice simple (mur / case libre) :

```
 TEST  génération du labyrinthe :
[[1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 1, 1, 0, 0, 1], [0, 0, 1, 1, 0, 0, 0,
1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0], [1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1,
0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1], [0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1,
0, 0, 0, 1, 1, 0, 1], [0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 0, 1],
[0, 1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1], [0, 1, 0, 1, 0, 0, 0,
0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 1], [1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0,
1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 1], [0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0,
1, 0, 1, 0, 0, 0, 1], [1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0],
[1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0], [1, 0, 1, 1, 0, 0, 1,
0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 1], [0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0,
0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0], [0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0,
0, 0, 0, 1, 0, 1, 0], [0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 1],
[1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1], [0, 1, 1, 0, 0, 1, 0,
1, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0], [1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0,
0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0], [1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0,
1, 0, 1, 1, 0, 0, 1], [0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0], [0, 1, 0, 0,
[0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0], [1, 0, 0, 1, 0, 1, 0,
0, 0, 1, 1, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 1], [0, 1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0,
1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1], [0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0,
0, 1, 0, 0, 0, 1, 0], [0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1], [1, 0, 1,
[1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1], [1, 0, 0, 0, 1, 1, 0,
0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1], [0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0,
```

Le labyrinthe est généré à l’aide d’un parcours en profondeur aléatoire. À partir d’une case de départ choisie au hasard, le chemin est étendu progressivement en explorant les cases voisines. Afin d’éviter la création de boucles, une règle anti-boucle est appliquée : une nouvelle case libre ne peut être ajoutée que si elle est voisine d’une seule case déjà libre. Cette contrainte permet d’obtenir des labyrinthes principalement composés de couloirs, ce qui facilite l’analyse des algorithmes de résolution .

La principale difficulté a été d'éviter la création de boucles tout en conservant un labyrinthe bien connecté. Cela a été résolu en ajoutant une règle anti-boucle qui limite l'ajout d'une nouvelle case libre à un seul voisin déjà libre, ce qui produit des labyrinthes structurés en couloirs.

Une règle anti-boucle est utilisée pour éviter les cycles.

L'utilisation d'une graine permet de générer exactement le même labyrinthe .

Test :

T1-vérifie que la génération est reproductible. Deux labyrinthes de même taille sont générés avec la même graine, puis comparés. Le résultat est positif, ce qui montre que la même graine produit exactement le même labyrinthe.

T2-mesure le temps de génération pour différentes tailles de labyrinthes. On observe que le temps augmente avec la taille, ce qui est normal, mais reste faible même pour des grilles plus grandes. La génération du labyrinthe n'est donc pas un facteur limitant pour la suite du projet.

```python
print(" TEST  génération du labyrinthe :  ")

lab1 = generer_labyrinthe(30, 30, graine=10)
lab2 = generer_labyrinthe(30, 30, graine=10)

print(lab1)
print(" ")

print("teste (graine=10) :", lab1 == lab2)
```

```
teste (graine=10) : True
```

```python
# Test — temps + tailles diffèrentes

for N in [5,20, 50, 500,]:
    t0 = time.time()
    lab = generer_labyrinthe(N, N, graine=1)
    t1 = time.time()
    print(f"N={N:2d} | temps={t1-t0:.4f}s ")
```

```
N= 5 | temps=0.0000s
N=20 | temps=0.0032s
N=50 | temps=0.0221s |
N=500 | temps=2.4880s
```

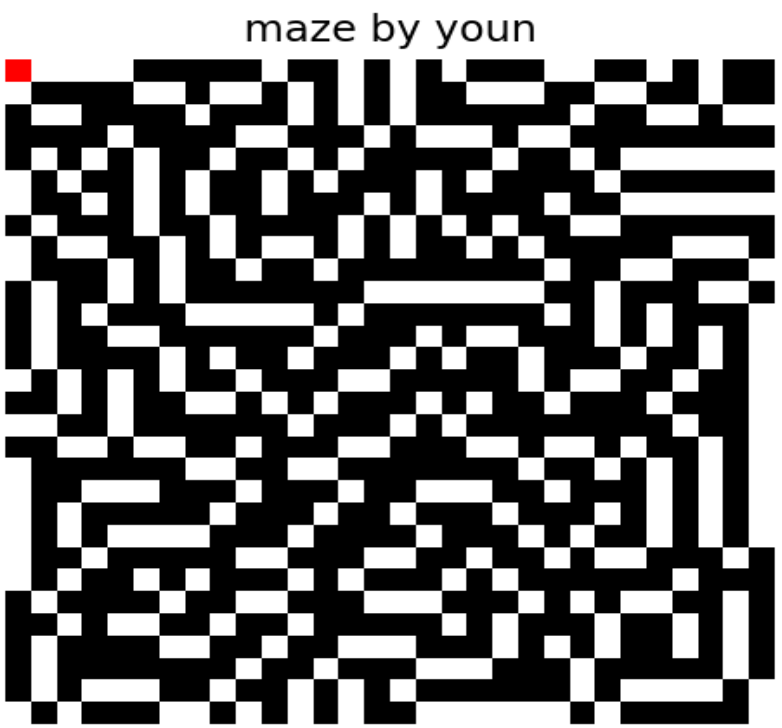
Une matrice pleine de 0 et de 1 n’est pas très agréable à lire. Pour rendre le labyrinthe plus compréhensible, on le transforme donc en image : les murs en noir, les cases libres en blanc, et le but en rouge. En un coup d’œil, on comprend beaucoup mieux la structure du labyrinthe

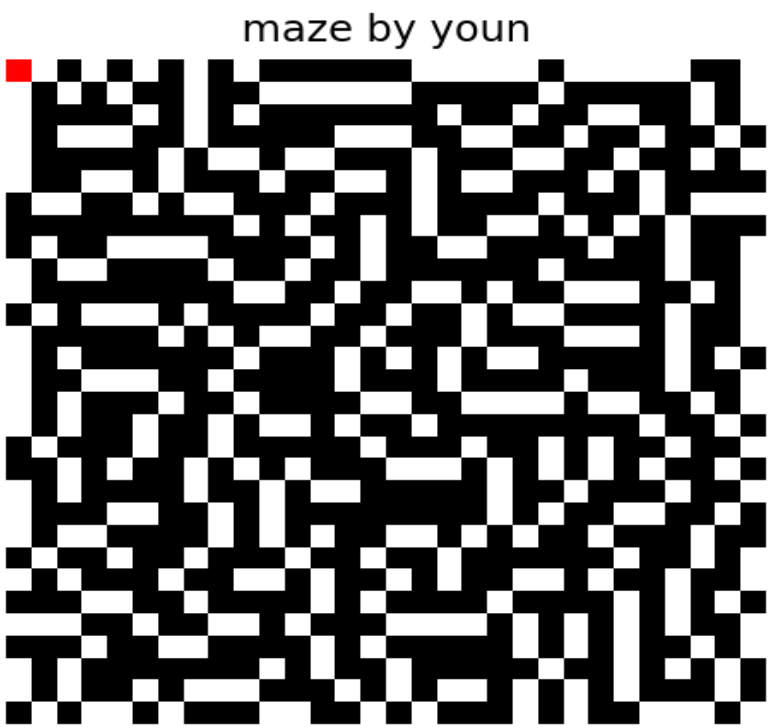
Tests:

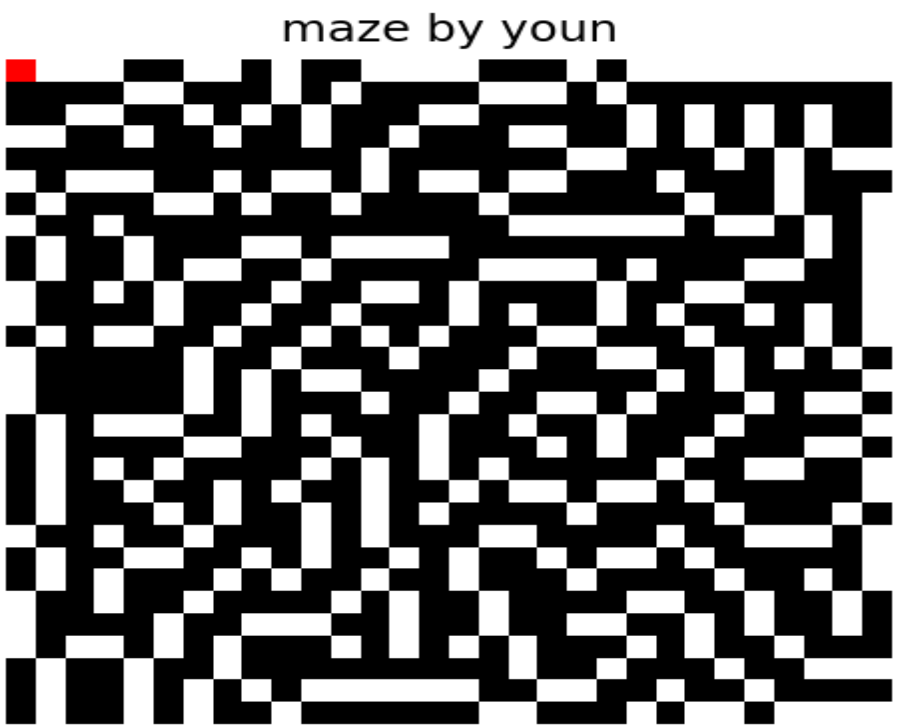
consiste à générer un labyrinthe 30×30 avec une graine différente et à afficher le but en rouge afin de vérifier que la visualisation du labyrinthe fonctionne correctement.

```python
for i in range(3):
    print(" ")
    print("Q2 :  labyrinthe ")
    print(" ")
    lab = generer_labyrinthe(30, 30, graine=i)

    but = (0, 0)
    print(" ")
    print("but rouge en : ", but)
    print(" ")
    afficher_labyrinthe(lab, but=but, titre=" maze by youn ")
```







SOUS_PROJET2 :

A pour objectif de résoudre le labyrinthe de manière optimale à l'aide d'un parcours en largeur (BFS). Une carte des distances est construite à partir du but, ce qui permet ensuite de reconstruire automatiquement le chemin le plus court depuis n'importe quelle case libre. Cette méthode sert de référence pour comparer les résultats obtenus avecl'algorithme génétique.

construire_carte_distances : tout simplement elle calcule la distance minimale entre chaque case libre et le but

Au début, le BFS peut donner des résultats faux (ou planter) si on ne gère pas les cas limites (but hors de la grille , but placé sur un mur, cases libres mais inatteignables)

```
print("TEST : construire_carte_distances (BFS) : ")

lab = generer_labyrinthe(20, 20, graine=0)
but = (3, 0) #si on le change on va tout changer   bien sûr
dist = construire_carte_distances(lab, but)

print(" ")

print(dist )

print(" ")
```

```
TEST : construire_carte_distances (BFS) :

[[-1, -1, 1, 0, -1, -1, -1, 5, -1, -1, -1, -1, 11, -1, 11, 12, 13, -1, 22, -1], [3, 2, -1, -1, 1,
2, -1, 4, -1, 6, 7, -1, -1, 10, -1, -1, -1, -1, 21, -1], [-1, -1, 3, -1, -1, -1, 3, -1, 5, -1, -1,
8, 9, -1, -1, -1, -1, 20, -1, 22], [6, -1, 4, -1, 13, -1, -1, -1, -1, 6, -1, -1, -1, 10, 11, -1,
19, -1, -1, 23], [-1, 5, -1, -1, 12, -1, 12, -1, 7, -1, -1, 12, 11, -1, -1, -1, -1, 18, -1, 24],
[6, -1, 6, -1, -1, 11, -1, -1, -1, 8, -1, -1, -1, 12, -1, -1, 17, -1, -1, 25], [7, -1, -1, 7, -1,
-1, 10, 11, -1, 9, -1, -1, -1, -1, -1, 16, -1, -1, -1, 26], [8, -1, -1, -1, 8, 9, -1, -1, -1, 10,
-1, 12, 13, 14, 15, -1, 17, -1, -1, 27], [9, -1, 10, 9, -1, -1, 10, -1, -1, -1, 11, -1, -1, -1, -1,
-1, 18, -1, 28, -1], [10, -1, -1, -1, -1, -1, -1, -1, 44, -1, -1, -1, 38, -1, 36, -1, -1, -1, 29,
-1], [11, -1, -1, -1, 48, -1, 46, 45, -1, 43, -1, 39, -1, 37, -1, 35, 34, -1, -1, 30], [-1, -1, 50,
49, -1, 47, -1, -1, -1, 42, -1, 40, -1, -1, -1, -1, -1, 33, -1, 31], [52, 51, -1, -1, -1, -1, -1,
-1, -1, -1, 41, -1, -1, -1, 75, -1, -1, -1, 32, -1], [-1, -1, 52, 53, 54, 55, -1, 57, 58, -1, -1,
-1, 77, 76, -1, 74, -1, -1, 33, -1], [54, 53, -1, -1, -1, -1, 56, -1, -1, 59, -1, -1, -1, -1, -1,
-1, 73, -1, 34, -1], [-1, -1, 54, -1, -1, -1, -1, -1, -1, -1, 60, 61, -1, 66, -1, -1, 72, -1, -1,
35], [62, -1, -1, 55, -1, 57, -1, 62, -1, 61, -1, -1, -1, 65, -1, -1, 71, -1, 36, -1], [61, -1, -1,
-1, 56, -1, -1, 61, -1, -1, 62, 63, 64, -1, 66, -1, 70, -1, 37, -1], [60, -1, 58, 57, -1, -1, 60,
-1, 62, -1, -1, -1, -1, -1, 67, -1, 69, -1, -1, -1], [-1, 59, -1, -1, 58, 59, -1, -1, -1, 63, 64,
65, 66, -1, -1, 68, -1, 70, 71, 72]]
```

Mais : la distance seule ne donne pas directement un chemin.

Une case peut : pointer vers un voisin invalide,ne pas savoir où aller ou créer des chemins incohérents impossibles à suivre.

construire_carte_directions :

Après le calcul de la carte des distances, l'objectif de cette étape est de transformer cette information globale en une information exploitable localement, afin de pouvoir reconstruire un chemin pas à pas depuis n'importe quelle case vers le but.

Reconstruction du chemin simplifiée (il suffit de suivre les directions).

Aucune ambiguïté possible : si une direction existe, elle est correcte.

Pour valider la construction de la carte de directions, j'ai généré un labyrinthe (30×30) puis j'ai choisi un but aléatoire sur une case libre. Après avoir calculé la carte des distances, j'ai construit la carte de directions et j'ai vérifié que la case du but est bien identifiée : sa distance doit être `0` et sa direction doit être `"GOAL"`. Ce test confirme que la carte directionnelle est cohérente et prête à être utilisée pour reconstruire un chemin

```python
# ============================================================
# TESTS -
# ============================================================

print(" construire_carte_directions")

lab = generer_labyrinthe(30, 30, graine=0)

# choisir un BUT qui est libre

but = case_libre_aleatoire(lab)

dist = construire_carte_distances(lab, but)

dirs = construire_carte_directions(dist)

print("But =", but, "| dist(but) =", dist[but[1]][but[0]], "| dir(but
```

```
 construire_carte_directions
But = (26, 14) | dist(but) = 0 | dir(but) = GOAL
```

reconstruire_chemin : comme indique le nom de la fonction on va construire un chemin depuis une case de départ jusqu'au but, en utilisant uniquement la carte de directions construite précédemment. Cette étape permet de passer d'une information abstraite (directions) à un chemin réel exploitable et visualisable.

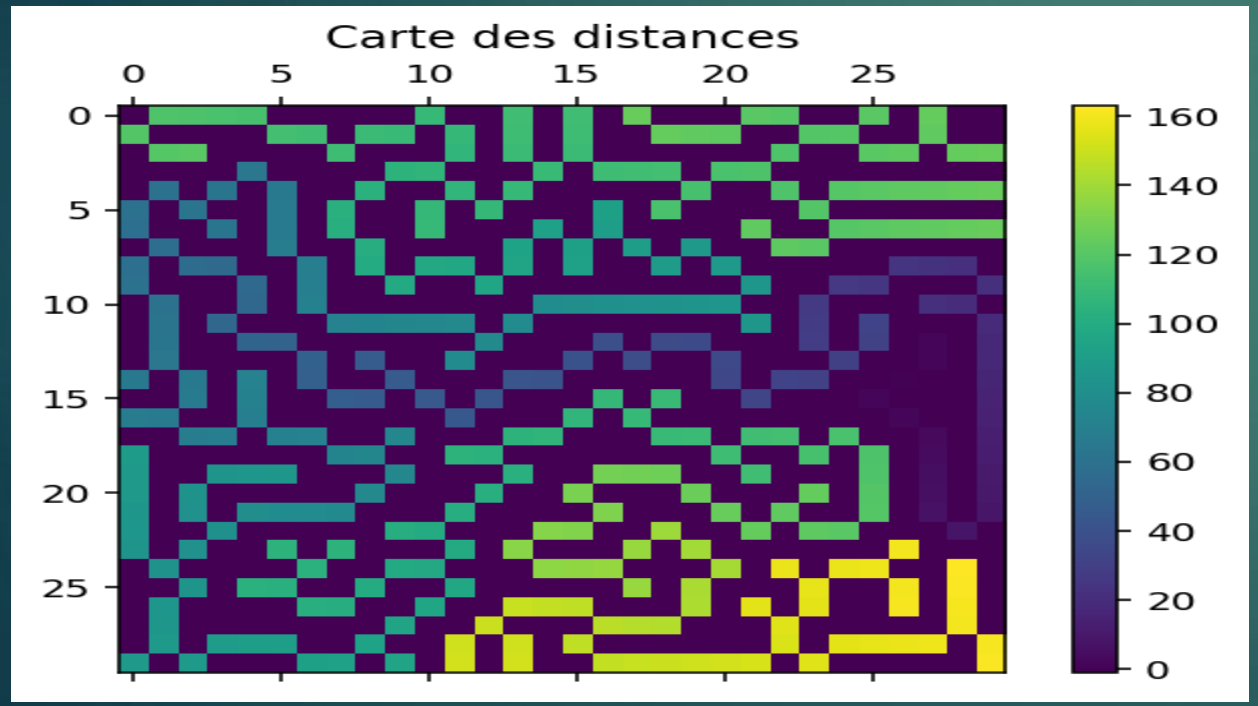
En pratique, certaines situations peuvent poser problème, comme l'absence de direction valide, un départ non atteignable ou le risque de boucles infinies si les directions sont mal suivies. Pour éviter ces situations, le chemin est reconstruit en suivant les directions pas à pas, avec un arrêt immédiat si aucune direction n'est disponible et une limite de sécurité pour garantir la terminaison. Cette méthode permet d'obtenir un chemin cohérent lorsqu'il existe, et de détecter proprement les cas où aucun chemin n'est possible.

Comme ce sous projet est mon sp préferé j'ai fait beacoup de tests (beacoup)

Ils sont tous dans le code je vais seulement les expliquer et analyser les résultats.

Bloc A : Carte de directions:

Ce test vérifie que la carte de directions est cohérente avec la carte des distances. Le but est bien identifié avec une distance nulle et une direction "GOAL", ce qui confirme que le BFS démarre correctement et que les directions sont bien construites à partir des distances.



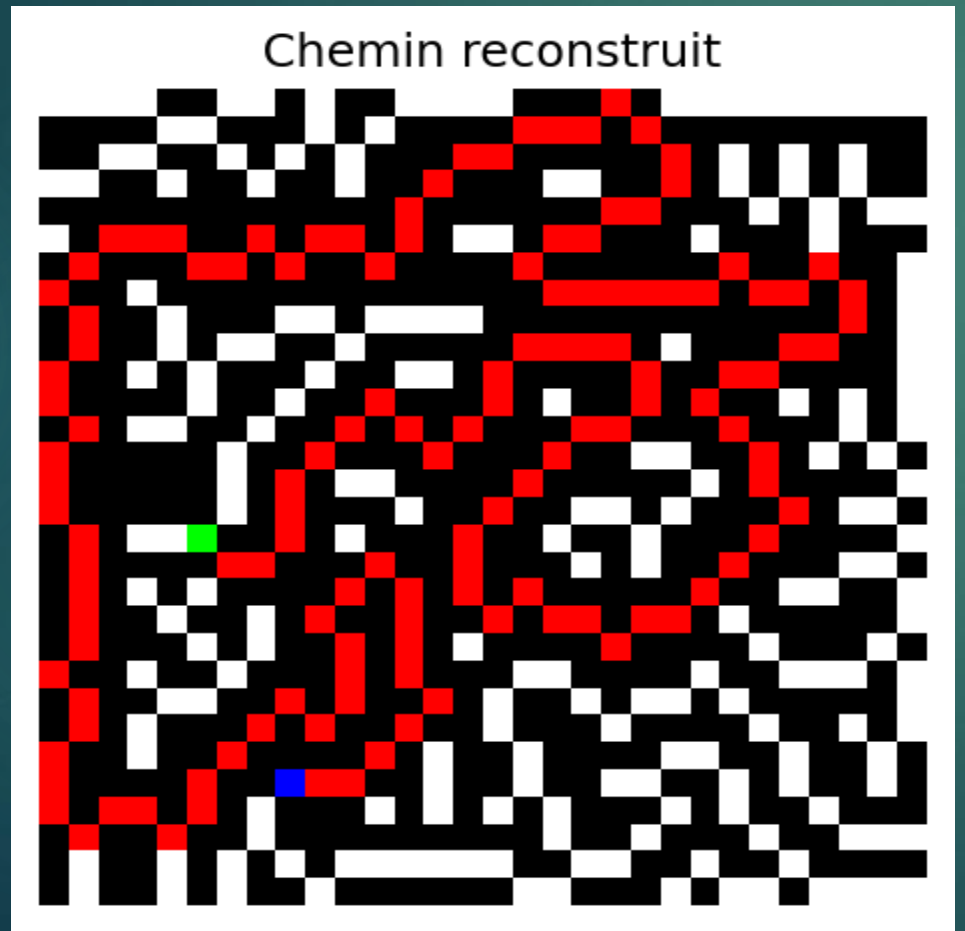

```
Bloc-A

 TEST  Q4 : affichage carte des distances

But = (26, 14) | distance au but = 0
```

Bloc B : Reconstruction du chemin

Ce test valide toute la chaîne de traitement, depuis le calcul des distances jusqu'au chemin final. Le fait que la longueur du chemin soit exactement égale à la distance calculée montre que le chemin reconstruit est optimal et correspond bien au plus court chemin.




```
Bloc-B
 TEST  chemin + distances

 Depart: (8, 25) | But: (5, 16)

 Distance depart -> but : 132

 Chemin trouvé | longueur = 132 | fin = (5, 16)
```

Bloc C : Influence de la taille du labyrinthe

Ce test étudie le comportement de la méthode lorsque la taille du labyrinthe augmente. On observe que la longueur moyenne des chemins et le temps de calcul augmentent avec N, ce qui est attendu, tout en conservant des résultats valides pour toutes les tailles testées.

```
Bloc-C
jeu de test fun : moyenne longueur solution vs N
(fight)
N=  8 | moy_len=12.50 | tests_validé=10/10 |
temps=0.00s
N= 16 | moy_len=50.70 | tests_validé=10/10 |
temps=0.02s
N= 32 | moy_len=131.10 | tests_validé=10/10 |
temps=0.07s
N= 64 | moy_len=529.50 | tests_validé=10/10 |
temps=0.29s
N=128 | moy_len=1541.70 | tests_validé=10/10 |
temps=1.08s
N=256 | moy_len=5647.90 | tests_validé=10/10 |
temps=4.23s
N=512 | moy_len=18145.60 | tests_validé=10/10 |
temps=18.44s
```

Bloc D : Validation du chemin

Ce dernier test vérifie automatiquement que le chemin trouvé est correct. Le résultat positif confirme que le chemin commence au bon endroit, arrive au but et respecte toutes les contraintes du labyrinthe.

```
Bloc-D
TEST  Q3 : validité du chemin reconstruit
Depart: (29, 27) But: (5, 16)
Chemin valide ? True
```

SOUS_PROJET3 : :)

On observe une augmentation du temps de calcul avec la taille, ce qui est attendu. La méthode reste utilisable sur toutes les tailles testées.

SOUS_PROJET3 : :)

Contrairement au BFS, qui donne toujours le chemin optimal mais nécessite une connaissance complète du labyrinthe, ce sous-projet explore une approche différente basée sur un algorithme génétique. L'objectif n'est pas forcément de trouver le chemin le plus court, mais de comprendre comment une population de solutions peut progressivement s'améliorer à partir d'essais, d'erreurs et de sélection.

Chaque individu représente une suite de déplacements dans le labyrinthe. Ces individus sont évalués, sélectionnés, croisés et mutés au fil des générations afin de produire des chemins de plus en plus efficaces.

Fonctions importantes et rôle global :

Simulation (simuler) :
Elle exécute réellement un individu dans le labyrinthe et permet de détecter les collisions, la progression et l'atteinte éventuelle du but.

Fitness (fitness) :
C'est le cœur de l'algorithme. Elle mesure la qualité d'un individu en combinant plusieurs critères simples mais complémentaires : distance au but, collisions, boucles, progression et atteinte de l'objectif.

Algorithme génétique (algorithme_genetique_simple) :
Il gère toute l'évolution : évaluation, sélection, croisement, mutation et conservation du meilleur individu global.

Phéromones :
Elles permettent de marquer les zones problématiques afin d'éviter que l'algorithme ne refasse indéfiniment les mêmes erreurs.(utile)

Même si ce sous-projet est intéréssant et que c'est la premier idée que j'ai eu pour une résolution , c’est aussi celui qui a posé le plus de difficultés. Contrairement aux méthodes déterministes comme le BFS, l’algorithme génétique demande beaucoup d’ajustements et d’expérimentations avant d’obtenir un comportement stable et cohérent. Cette partie du projet a donc nécessité de nombreux tests, essais et corrections.

Difficulté : comprendre pourquoi l’algorithme ne progresse pas
Au début, l’algorithme génétique s’exécutait correctement , mais les résultats étaient mauvais la fitness ne baissait presque pas et les individus adoptaient des comportements incohérents (collisions rapides, boucles, déplacements inutiles). Le principal problème était qu’il était difficile de comprendre ce qui empêchait réellement l’amélioration.

Pour mieux comprendre le comportement de l’algorithme,La fonction simuler permet d’observer précisément le chemin suivi, les collisions et la position finale, tandis que la fonction fitness traduit ces informations en score. Cette séparation a permis d’identifier plus facilement les causes des mauvais résultats et d’ajuster progressivement les pénalités.
(c'était un petit mélange de coef)

Une autre difficulté : gérer correctement les collisions et les arrêts précoces
Un problème fréquent était que de nombreux individus entraient en collision très tôt (mur ou sortie de la grille), ce qui rendait leur évaluation peu informative. Sans gestion claire, ces cas perturbaient l'évolution de la population.

La simulation a été conçue pour détecter immédiatement les collisions et arrêter proprement l'exécution de l'individu. Une pénalité forte est ensuite appliquée dans la fitness. Cette gestion explicite des collisions a stabilisé l'évaluation des individus et rendu la sélection plus cohérente

Une

Une troisième difficulté : éviter les boucles et déplacements inutiles
Un autre problème technique observé était la présence de boucles , certains individus repassaient plusieurs fois sur les mêmes cases, ce qui donnait des chemins longs mais inutiles. Sans pénalité adaptée, ces comportements pouvaient être favorisés par erreur.

Une pénalité basée sur la différence entre la longueur du chemin et le nombre de cases uniques visitées a été ajoutée. Cette solution simple permet de détecter efficacement les boucles sans avoir à stocker d’informations complexes.

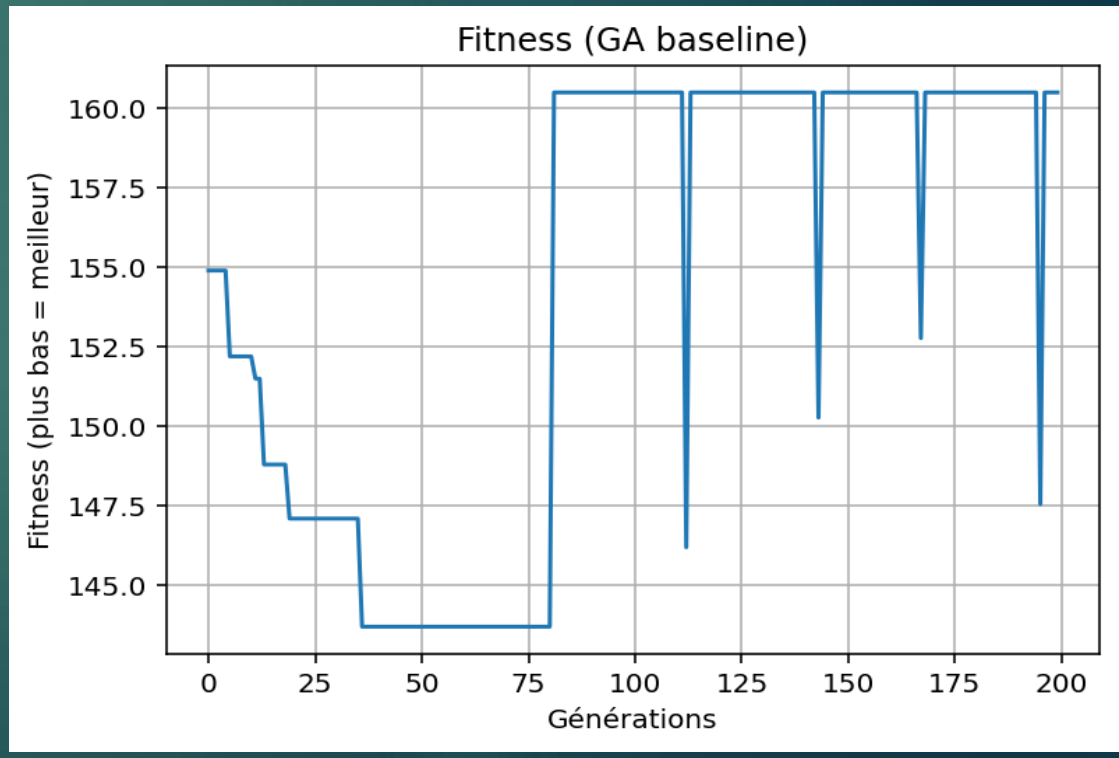
D’autres problèmes sont apparus en cours de route, mais ils ont été corrigés (j'éspère) petit à petit grâce aux tests. On se concentre maintenant sur les résultats obtenus, qui montrent que la méthode fonctionne correctement dans sa version finale.

On fixe un labyrinthe de référence (même taille + même graine) puis on calcule le chemin optimal BFS, qui servira de base de comparaison.

```
In [1]: %runfile C:/Users/artyo/Downloads/
younes_maze_projet.py --wdir
TEST SP3
Lab = 40 x 40 | graine = 7
```

```
TEST A : GA baseline (avec phéromones)
GA terminé sans atteindre le but. | meilleur fitness = 143.7 | gen = 36 | atteint = False
Fitness début = 154.9 | fin = 160.5 | min = 143.7
GA : chemin renvoyé | longueur = 9 | fin = (26, 6)
```

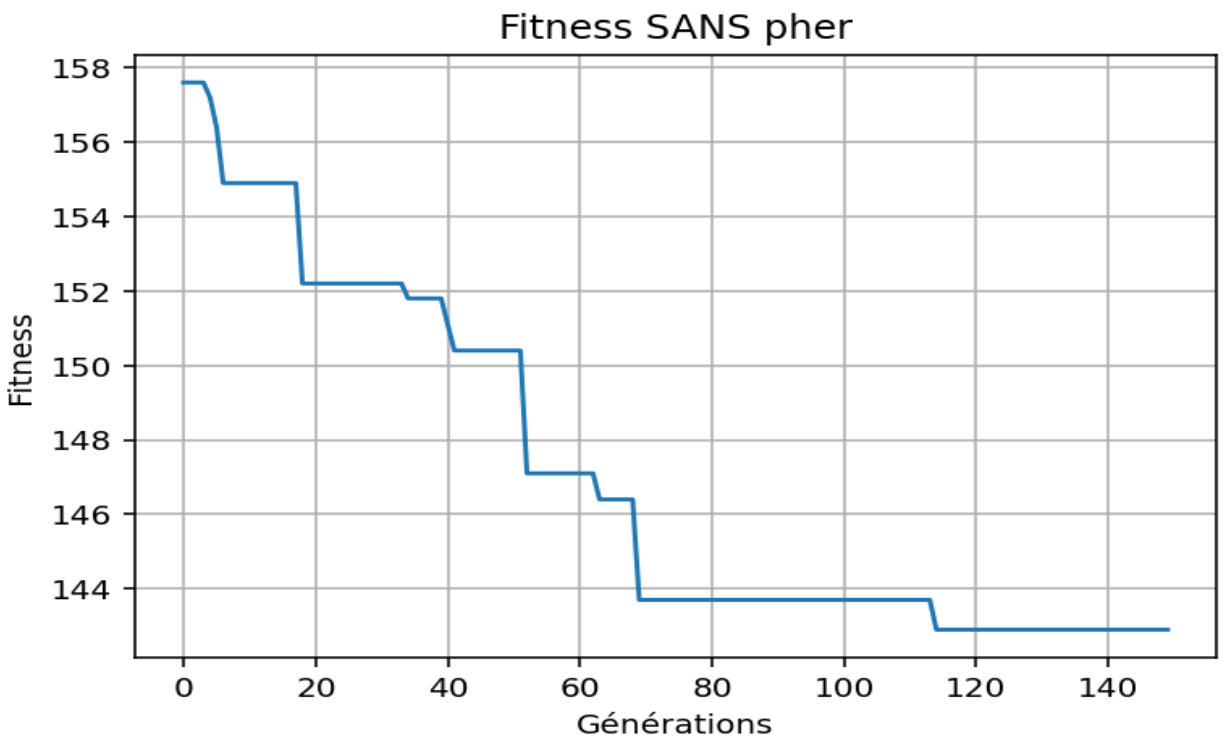
TEST-A : montrer que le GA progresse (fitness qui baisse) + visualiser le meilleur chemin.La courbe de fitness montre une amélioration rapide au début, ce qui indique que l'algorithme apprend à éviter les comportements les plus pénalisants. La stagnation observée par la suite, avec quelques améliorations ponctuelles, est typique des algorithmes génétiques et reflète la difficulté du problème ainsi que les limites de la configuration choisie.

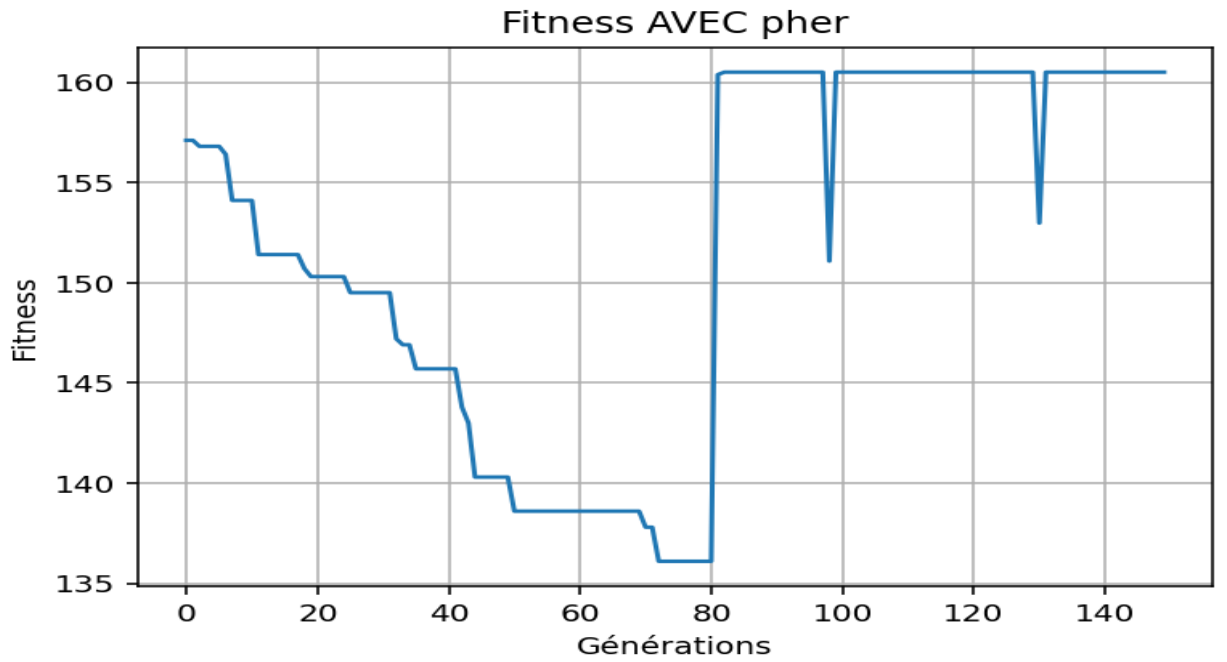


Test B : Effet des phéromones (avec vs sans)

Sans phéromones, la fitness diminue de manière progressive puis se stabilise, ce qui montre que l'algorithme génétique apprend, mais finit par stagner autour d'une solution moyenne. La convergence est relativement stable, mais l'exploration reste limitée.

Avec phéromones, on observe une meilleure amélioration au début, avec une fitness plus basse atteinte plus rapidement. En revanche, l'activation des phéromones introduit des variations plus fortes par la suite, dues à une exploration plus agressive de nouvelles zones du labyrinthe.





Les phéromones permettent de sortir plus facilement de la stagnation et d'explorer davantage, mais au prix d'une convergence moins stable. Elles améliorent le potentiel de recherche sans garantir systématiquement une meilleure solution finale.

```
 TEST B : GA avec vs sans phéromones
GA terminé sans atteindre le but. | meilleur fitness = 142.9 | gen = 114 | atteint = False
SANS pher | fitness_fin = 142.9 | min = 142.9 | chemin_recu = True
Courbe sauvegardée : fitness_SANS_pher.png
GA terminé sans atteindre le but. | meilleur fitness = 136.1 | gen = 72 | atteint = False
AVEC pher | fitness_fin = 160.5 | min = 136.1 | chemin_recu = True
```

Test C : Influence des paramètres :Dans ce test précis, la configuration plus petite obtient la meilleure fitness. Ça montre que les paramètres n'ont pas un effet 'toujours pareil' : selon le labyrinthe et l'aléatoire, certaines configs marchent mieux, donc il faut tester et choisir un compromis.

```
TEST C : Variations paramètres (3 cas)
GA terminé sans atteindre le but. | meilleur fitness = 132.9 | gen = 67 | atteint = False
Plus petit | N= 60 L=100 TS=0.3 TM=0.15 | fin=132.9 | min=132.9
GA terminé sans atteindre le but. | meilleur fitness = 135.7 | gen = 149 | atteint = False
Baseline   | N= 80 L=150 TS=0.3 TM=0.15 | fin=135.7 | min=135.7
GA terminé sans atteindre le but. | meilleur fitness = 144.1 | gen = 144 | atteint = False
Plus grand | N=120 L=200 TS=0.3 TM=0.15 | fin=144.1 | min=144.1
FIN de projet (finalement)
C'était Fun , merci beaucoup
```

Ce test étudie l'impact de la taille de la population et de la longueur des individus sur les performances de l'algorithme génétique. Les résultats montrent qu'une configuration trop petite converge rapidement mais reste limitée, tandis qu'une configuration plus grande explore davantage mais converge plus lentement et avec un coût de calcul plus élevé. La configuration intermédiaire (baseline) offre un bon compromis entre qualité de la solution et stabilité, même si aucune configuration ne garantit l'atteinte du but.

Ce test étudie l'impact de la taille de la population et de la longueur des individus sur les performances de l'algorithme génétique. Les résultats montrent qu'une configuration trop petite converge rapidement mais reste limitée, tandis qu'une configuration plus grande explore davantage mais converge plus lentement et avec un coût de calcul plus élevé. La configuration intermédiaire (baseline) offre un bon compromis entre qualité de la solution et stabilité, même si aucune configuration ne garantit l'atteinte du but.

Bonus : (je n'avais pas assez de temps pour le faire)

Pistes d’optimisation et complexité.
L’algorithme génétique utilisé reste coûteux en temps de calcul, car chaque génération nécessite la simulation d’un grand nombre d’individus. Des optimisations auraient pu être envisagées, comme l’adaptation dynamique des paramètres (taille de population, taux de mutation) ou une gestion plus fine des phéromones pour accélérer la convergence. Par manque de temps, ces améliorations n’ont pas été implémentées, mais elles constituent des pistes intéressantes pour améliorer les performances sans modifier le principe général de l’algorithme.

Comparaison avec le BFS optimal.
Le BFS a été utilisé comme solution de référence, car il fournit un chemin optimal par construction. Une comparaison plus approfondie aurait pu être menée en analysant systématiquement la différence entre les chemins produits par le BFS et ceux obtenus par l’algorithme génétique, à la fois en longueur et en structure. Faute de temps, cette comparaison est restée principalement qualitative, mais les visualisations montrent clairement l’écart entre une méthode optimale et une méthode d’optimisation heuristique.

Et voilà ! Après quelques semaines de travail intense (et de cafés bien mérités), nous avons réussi à générer et explorer des labyrinthes, créer des cartes de distances, et trouver des chemins optimaux pour atteindre notre but. Chaque étape était un défi passionnant, entre la construction de la grille, la recherche du départ, et l'analyse des différents algorithmes de recherche. Il y a eu des moments où on a dû réajuster notre code, mais chaque bug corrigé était une petite victoire. Ce projet a prouvé qu'avec un peu de logique, d'imagination, et beaucoup de persévérance, on peut transformer des matrices en véritables aventures . Merci beacoup
Younes Nassih